

18 — Deployment & Serving: From Notebook to Production

Time: ~5-6 hours | **Level:** Professional

What you'll learn:

- FastAPI: building production-grade model APIs
- Docker: containerizing ML applications
- Model serving patterns: online, batch, streaming
- Quantization & ONNX: making models fast
- Load testing: knowing your system's limits
- Kubernetes basics: scaling model serving
- vLLM: high-throughput LLM serving

Prerequisites: Notebook 10 (Production fine-tuning), Notebook 17 (MLOps basics)

The Deployment Gap

A model in a notebook is a **science experiment**. A model behind an API with monitoring, scaling, and rollback is an **engineering system**.

Most ML projects die in the deployment gap. This notebook ensures yours don't.

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json
import time
import os
from pathlib import Path

sns.set_theme(style='whitegrid', font_scale=1.1)
```

1. FastAPI — The Modern Python API Framework

Why FastAPI (not Flask):

Feature	Flask	FastAPI
Speed	Slow (WSGI)	Fast (ASGI, async)
Type checking	Manual	Automatic (Pydantic)

Feature	Flask	FastAPI
Docs	Manual	Auto-generated (Swagger UI)
Validation	Manual	Automatic
Async support	Plugin	Native

Model serving API structure:

```

POST /predict          → single prediction
POST /predict/batch    → batch predictions
GET /health            → liveness check
GET /model/info        → model metadata

```

```

In [ ]: # — FastAPI model serving app —————
# This is the code you'd put in `app.py`

fastapi_code = '''
"""Production model serving API."""
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import numpy as np
import joblib
import time
import logging

# — Setup —————
app = FastAPI(
    title="ML Model API",
    description="Production model serving with FastAPI",
    version="1.0.0",
)

logger = logging.getLogger(__name__)

# Load model at startup (not per request!)
model = None
model_metadata = {}

@app.on_event("startup")
async def load_model():
    global model, model_metadata
    model = joblib.load("model.joblib")
    model_metadata = {
        "model_type": type(model).__name__,
        "loaded_at": time.time(),
    }
    logger.info(f"Model loaded: {model_metadata}")

# — Request/Response schemas —————
class PredictionRequest(BaseModel):
    features: list[float] = Field(..., min_length=1, description="Input feat

class Config:

```

```

        json_schema_extra = {
            "example": {"features": [0.5, 1.2, -0.3, 2.1]}
        }

class PredictionResponse(BaseModel):
    prediction: int
    probability: float
    latency_ms: float

class BatchRequest(BaseModel):
    instances: list[list[float]]

class HealthResponse(BaseModel):
    status: str
    model_loaded: bool

# — Endpoints —————
@app.get("/health", response_model=HealthResponse)
async def health():
    return HealthResponse(status="healthy", model_loaded=model is not None)

@app.get("/model/info")
async def model_info():
    return model_metadata

@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    if model is None:
        raise HTTPException(status_code=503, detail="Model not loaded")

    start = time.time()
    features = np.array(request.features).reshape(1, -1)
    prediction = int(model.predict(features)[0])
    probability = float(model.predict_proba(features).max())
    latency_ms = (time.time() - start) * 1000

    return PredictionResponse(
        prediction=prediction,
        probability=probability,
        latency_ms=latency_ms,
    )

@app.post("/predict/batch")
async def predict_batch(request: BatchRequest):
    if model is None:
        raise HTTPException(status_code=503, detail="Model not loaded")

    start = time.time()
    features = np.array(request.instances)
    predictions = model.predict(features).tolist()
    probabilities = model.predict_proba(features).max(axis=1).tolist()
    latency_ms = (time.time() - start) * 1000

    return {
        "predictions": predictions,
        "probabilities": probabilities,
    }

```

```

        "count": len(predictions),
        "latency_ms": latency_ms,
    }

    ...

print(fastapi_code)
print("\n" + "="*60)
print("To run: uvicorn app:app --host 0.0.0.0 --port 8000")
print("Docs at: http://localhost:8000/docs (auto-generated Swagger UI)")
print("="*60)

```

2. Docker — Reproducible Deployments

Why Docker:

- "It works on my machine" → "It works **everywhere** the same way"
- Packages: code + dependencies + OS libraries into a single image
- Same image runs in: local dev, CI/CD, staging, production

Docker concepts:

Concept	Analogy	Purpose
Dockerfile	Recipe	Describes how to build the image
Image	Template	Immutable snapshot of your app
Container	Running instance	An image that's actually executing
Registry	App store	Where images are stored (Docker Hub, ECR)

```

In [ ]: # — Production Dockerfile for ML serving —————

dockerfile = '''
# — Stage 1: Build dependencies —————
FROM python:3.10-slim as builder

WORKDIR /app

# Install dependencies first (cached if requirements.txt unchanged)
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

# — Stage 2: Production image —————
FROM python:3.10-slim

WORKDIR /app

# Copy installed packages from builder
COPY --from=builder /root/.local /root/.local
ENV PATH=/root/.local/bin:$PATH

# Copy application code

```

```

COPY app.py .
COPY model.joblib .

# Non-root user for security
RUN useradd --create-home appuser
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \\\
    CMD curl -f http://localhost:8000/health || exit 1

EXPOSE 8000

# Production ASGI server with multiple workers
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000", "--workers",
    ...

print("# Dockerfile for ML Model Serving")
print(dockerfile)

# requirements.txt
requirements = """fastapi==0.104.1
uvicorn[standard]==0.24.0
numpy==1.26.2
scikit-learn==1.3.2
joblib==1.3.2
pydantic==2.5.0
"""

print("\n# requirements.txt")
print(requirements)

print("\n# Build and run:")
print("docker build -t ml-api:v1 .")
print("docker run -p 8000:8000 ml-api:v1")

```

3. Model Serving Patterns

Pattern	Latency	Throughput	Use Case
Online (REST/gRPC)	<100ms	Medium	Real-time predictions (search, recommendations)
Batch (scheduled)	Hours	Very high	Periodic scoring (email campaigns, risk scores)
Streaming (Kafka/Flink)	<1s	High	Event-driven (fraud detection, anomaly detection)
Edge (on-device)	<10ms	N/A	Mobile, IoT, offline

```

In [ ]: # — Serving patterns comparison —————

patterns = {
    'Online\n(FastAPI)': {'latency_p50_ms': 5, 'throughput_rps': 1000},

```

```

'Batch\n(Spark)': {'latency_p50_ms': 3600000, 'throughput_rps': 100000},
'Streaming\n(Kafka)': {'latency_p50_ms': 100, 'throughput_rps': 10000},
'Edge\n(ONNX)': {'latency_p50_ms': 2, 'throughput_rps': 100},
}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

names = list(patterns.keys())
p50 = [patterns[n]['latency_p50_ms'] for n in names]
throughput = [patterns[n]['throughput_rps'] for n in names]

axes[0].barh(names, [np.log10(p + 1) for p in p50], color='steelblue')
axes[0].set_xlabel('log10(Latency in ms)')
axes[0].set_title('Latency (lower = faster)')

axes[1].barh(names, [np.log10(t) for t in throughput], color='coral')
axes[1].set_xlabel('log10(Requests per second)')
axes[1].set_title('Throughput (higher = more capacity)')

plt.suptitle('Model Serving Patterns: Latency vs Throughput Tradeoffs', font
plt.tight_layout()
plt.show()

```

4. Model Optimization — Making Models Fast

Techniques (ordered by effort):

Technique	Speedup	Quality Loss	Effort
ONNX export	2-3x	None	Low
Quantization (INT8)	2-4x	Minimal	Low
Pruning	1.5-3x	Small	Medium
Knowledge distillation	3-10x	Small	High

```

In [ ]: # — ONNX export and quantization —————
import torch
import torch.nn as nn

# Simple model for demonstration
class SimpleClassifier(nn.Module):
    def __init__(self, input_dim=20, hidden_dim=64, num_classes=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, num_classes),
        )
    def forward(self, x):
        return self.net(x)

```

```

model = SimpleClassifier()
model.eval()

# Export to ONNX
dummy_input = torch.randn(1, 20)
onnx_path = "/tmp/model.onnx"

torch.onnx.export(
    model, dummy_input, onnx_path,
    input_names=['features'],
    output_names=['logits'],
    dynamic_axes={'features': {0: 'batch_size'}, 'logits': {0: 'batch_size'}},
    opset_version=17,
)
print(f"ONNX model exported to {onnx_path}")
print(f"ONNX file size: {os.path.getsize(onnx_path) / 1024:.1f} KB")

# Quantization: INT8
model_int8 = torch.quantization.quantize_dynamic(
    model, {nn.Linear}, dtype=torch.qint8
)

import tempfile
fp32_path = os.path.join(tempfile.gettempdir(), 'model_fp32.pt')
int8_path = os.path.join(tempfile.gettempdir(), 'model_int8.pt')
torch.save(model.state_dict(), fp32_path)
torch.save(model_int8.state_dict(), int8_path)

fp32_size = os.path.getsize(fp32_path) / 1024
int8_size = os.path.getsize(int8_path) / 1024

print(f"\nFP32 model: {fp32_size:.1f} KB")
print(f"INT8 model: {int8_size:.1f} KB")
print(f"Compression: {fp32_size/int8_size:.1f}x smaller")

```

5. Load Testing — Know Your Limits

Never deploy without knowing:

- Maximum requests per second (RPS)
- P50, P95, P99 latency
- At what load does the system break?

```

In [ ]: # — Simulate load test results —————

# Simulate latency under increasing load
np.random.seed(42)
user_counts = [10, 25, 50, 100, 200, 500, 1000]

latencies = {}
for users in user_counts:
    base = 5 # ms

```

```

    load_factor = max(0, (users - 100) * 0.5)
    samples = base + load_factor + np.random.exponential(2 + load_factor * 0.5)
    latencies[users] = samples

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Latency distribution at different loads
for users in [10, 100, 500]:
    axes[0].hist(latencies[users], bins=50, alpha=0.5, label=f'{users} users')
axes[0].set_xlabel('Latency (ms)')
axes[0].set_title('Latency distribution')
axes[0].legend()
axes[0].set_xlim(0, 100)

# Percentile latencies vs load
for pct, name in [(50, 'P50'), (95, 'P95'), (99, 'P99')]:
    values = [np.percentile(latencies[u], pct) for u in user_counts]
    axes[1].plot(user_counts, values, 'o-', label=name)
axes[1].set_xlabel('Concurrent Users')
axes[1].set_ylabel('Latency (ms)')
axes[1].set_title('Latency percentiles vs load')
axes[1].legend()

# Throughput
throughputs = [min(u * 10, 2000) + np.random.normal(0, 50) for u in user_counts]
axes[2].plot(user_counts, throughputs, 'go-', linewidth=2)
axes[2].axhline(y=2000, color='r', linestyle='--', label='Server capacity')
axes[2].set_xlabel('Concurrent Users')
axes[2].set_ylabel('Requests/sec')
axes[2].set_title('Throughput (saturates at capacity)')
axes[2].legend()

plt.suptitle("Load Testing Results: Understanding Your System's Limits", fontweight='bold')
plt.tight_layout()
plt.show()

```

6. Kubernetes & LLM Serving

K8s for ML:

- **Pods:** Run your container
- **Deployment:** Manages replicas, rolling updates
- **Service:** Load balances across pods
- **HPA:** Auto-scales on CPU/memory/custom metrics

LLM Serving with vLLM:

- **PagedAttention:** Efficient KV cache management
- **Continuous batching:** No waiting for batch completion
- **2-24x faster** than naive PyTorch inference

```
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3.1-8B-Instruct \
    --tensor-parallel-size 2
```

```
In [ ]: # — LLM Serving comparison —————

serving_options = {
    'PyTorch\n(naive)': {'throughput': 30, 'latency_ms': 200},
    'HuggingFace\nTGI': {'throughput': 150, 'latency_ms': 100},
    'vLLM': {'throughput': 300, 'latency_ms': 80},
    'TensorRT-\nLLM': {'throughput': 400, 'latency_ms': 50},
}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

names = list(serving_options.keys())
throughputs = [serving_options[n]['throughput'] for n in names]
latencies_llm = [serving_options[n]['latency_ms'] for n in names]
colors = ['#ff6b6b', '#feca57', '#48dbfb', '#0abde3']

axes[0].bar(names, throughputs, color=colors)
axes[0].set_ylabel('Tokens/second')
axes[0].set_title('LLM Serving Throughput (higher = better)')

axes[1].bar(names, latencies_llm, color=colors)
axes[1].set_ylabel('Time to first token (ms)')
axes[1].set_title('Latency (lower = better)')

plt.suptitle('LLM Serving Frameworks: 7B model on A100 GPU (typical)', fontsize=12)
plt.tight_layout()
plt.show()
```

Key Takeaways

Concept	One-Line Summary
FastAPI	The modern standard for ML APIs — async, typed, auto-documented
Docker	Package everything → deploy anywhere consistently
ONNX	Export once, run anywhere, 2-3x faster than PyTorch
Quantization	INT8 = 2-4x faster, minimal accuracy loss
Load testing	Know your P99 latency and max RPS before launch
Kubernetes	Auto-scale your API from 2 to 100 pods based on load
vLLM	The standard for LLM serving — PagedAttention = game changer

What to study next:

- **Notebook 19:** System Design for AI (putting it all together)
- **Notebook 20:** Capstone Project (build a complete deployed system)